

Product Safety and Inspection Management System with Retrieval-Augmented Generation (Track A)

Final Project Report

Given Kangwa – ZDA24B027
Kelvin Kimani – ZDA24B019

Contents

1	Introduction	2
2	Project Objectives	2
3	System Overview	2
4	Database Design	3
5	Dataset Description	3
6	Key SQL Queries	4
6.1	Query 1: Product–Inspection Join	4
6.2	Query 2: Inspection Ranking	4
6.3	Query 3: Certification–Inspection Join	4
7	Performance Analysis and Optimization	5
7.1	Implemented Indexes	5
7.2	Performance Results	5
8	Trigger Implementation	5
8.1	Recall Logging Trigger	5
8.2	Workflow	6
9	Retrieval-Augmented Generation (RAG) Implementation	6
9.1	RAG Pipeline	6
9.2	Pinecone Configuration	6
10	Natural Language Retrieval Demonstration	7
11	Limitations	7
12	Future Work	7
13	Conclusion	8
14	AI Usage Disclosure	8

1. Introduction

Ensuring product safety and regulatory compliance is essential for protecting consumers and maintaining trust in markets. Organizations involved in manufacturing and distribution are required to track product certifications, conduct inspections, and manage product recalls efficiently. Traditional database systems are effective at storing structured records; however, retrieving information through natural language remains challenging.

This project presents the development of a Product Safety and Inspection Management System enhanced with Retrieval-Augmented Generation (RAG) capabilities. The system integrates a relational database implemented in PostgreSQL with semantic search powered by Sentence Transformers and Pinecone. The resulting system supports both conventional SQL-based retrieval and natural language querying of product information.

2. Project Objectives

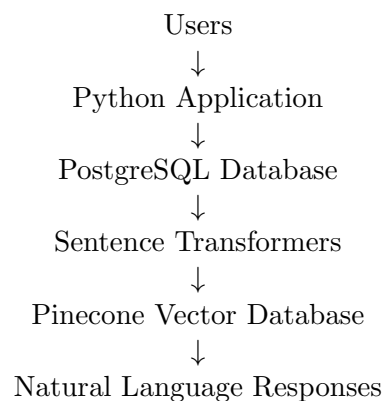
The objectives of this project were:

- To design and implement a normalized relational database for managing product safety information.
- To populate the database with realistic product safety records.
- To perform analytical queries using SQL.
- To optimize database performance using indexes.
- To implement database automation using triggers.
- To develop a Retrieval-Augmented Generation (RAG) component capable of answering natural language questions using vector similarity search.

3. System Overview

The Product Safety and Inspection Management System combines traditional database technologies with modern AI techniques.

The relational database manages structured information such as products, inspections, certifications, and recalls. The RAG component enables users to ask questions in natural language and retrieve semantically relevant results.



4. Database Design

The database was designed using principles of normalization to ensure data consistency and minimize redundancy.

The final schema consisted of the following tables:

Categories

Stores the different categories of products.

Products

Stores product details and descriptions.

Inspections

Records inspection activities and inspection outcomes.

Certifications

Stores certification status, expiry dates, issuing authorities, and related notes.

Recalls

Stores recall information related to products.

Recall Log

Stores automated logs of recall insertions.

The database schema was normalized to Third Normal Form (3NF). This design reduced redundancy and improved maintainability by ensuring that each non-key attribute depended only on the primary key.

5. Dataset Description

Synthetic datasets were generated to simulate realistic product safety scenarios involving inspections, certifications, and recalls.

The final database consisted of the following records:

Table 1: Record Counts by Table

Table	Number of Records
Products	1000
Inspections	2038
Certifications	1000
Recalls	297
Categories	8

The dataset included examples of:

- Valid certifications,
- Expired certifications,
- Failed inspections,
- Recalled products,
- Regulatory compliance notes.

These scenarios enabled meaningful testing of both SQL queries and semantic retrieval.

6. Key SQL Queries

6.1 Query 1: Product–Inspection Join

Purpose

To retrieve inspection information associated with products.

SQL Query

```
SELECT i.product_id ,
       product_name ,
       manufacturer ,
       inspection_id ,
       inspection_date ,
       failure_reason ,
       notes
FROM products p
JOIN inspections i
ON p.product_id = i.product_id;
```

Discussion

This query demonstrates the use of joins to combine product information with inspection records.

6.2 Query 2: Inspection Ranking

Purpose

To rank products according to the number of inspections performed.

SQL Query

```
SELECT product_id ,
       COUNT(*) AS total_inspections ,
       RANK() OVER (
           ORDER BY COUNT(*) DESC
       ) AS inspection_rank
FROM inspections
GROUP BY product_id;
```

Discussion

This query demonstrates the use of aggregation and window functions for analytical reporting.

6.3 Query 3: Certification–Inspection Join

Purpose

To analyze relationships between certifications and inspections.

SQL Query

```
SELECT certification_id ,
       expiry_date ,
       inspection_id ,
       failure_reason
FROM inspections i
JOIN certifications c
ON i.product_id = c.product_id;
```

Discussion

This query combines certification information with inspection outcomes.

7. Performance Analysis and Optimization

Database performance was evaluated using PostgreSQL's `EXPLAIN ANALYZE` functionality.

Indexes were introduced to optimize frequently used queries.

7.1 Implemented Indexes

```
CREATE INDEX idx_product_id
ON inspections(product_id);

CREATE INDEX idx_inspections_product_rank
ON inspections(product_id);

CREATE INDEX idx_certifications_product
ON certifications(product_id);
```

7.2 Performance Results

Table 2: Query Performance Before and After Indexing

Query	Before Index (ms)	After Index (ms)
Product-Inspection Join	1.948	0.844
Inspection Ranking	0.697	0.705
Certification-Inspection Join	0.828	0.592

Discussion

The first and third queries showed noticeable improvements following index creation. However, the ranking query exhibited minimal benefit due to the relatively small dataset size and the nature of aggregation operations.

These findings indicate that indexing becomes increasingly valuable as dataset sizes grow.

8. Trigger Implementation

Database triggers were used to automate recall logging.

8.1 Recall Logging Trigger

A trigger was implemented to automatically record newly inserted recalls into a dedicated logging table.

Recall Log Table

```
CREATE TABLE recall_log (
  log_id SERIAL PRIMARY KEY,
  recall_id INT,
  product_id INT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

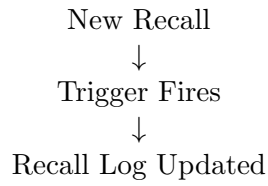
Trigger Function

The trigger function inserts information into the recall log whenever a new recall is added.

Trigger

The trigger executes automatically after recall insertions.

8.2 Workflow



This functionality improves traceability and auditing of recall events.

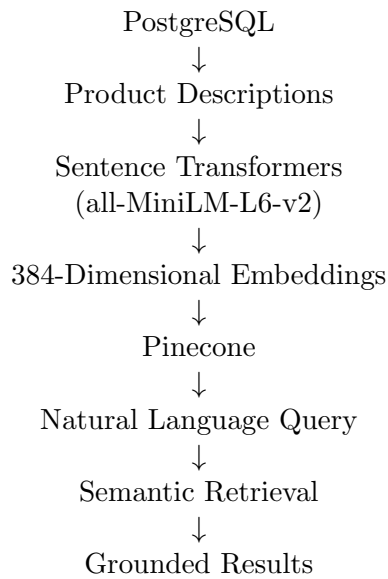
9. Retrieval-Augmented Generation (RAG) Implementation

A RAG component was integrated to support natural language retrieval.

The implementation used:

- Sentence Transformers,
- Pinecone,
- PostgreSQL,
- Python.

9.1 RAG Pipeline



9.2 Pinecone Configuration

Table 3: Pinecone Index Configuration

Parameter	Value
Index Name	product-safety-rag
Dimension	384
Similarity Metric	Cosine
Stored Records	1000

Sentence Transformers converted product descriptions into 384-dimensional embeddings, which were subsequently uploaded to Pinecone.

Pinecone acted as the vector database responsible for retrieving semantically similar records.

10. Natural Language Retrieval Demonstration

The RAG application allowed users to ask questions without writing SQL.

Example questions included:

Example 1

Which products have certification concerns?

Example 2

Which products mention traceability documentation?

Example 3

Find products related to regulatory compliance.

The system returned:

- Product names,
- Descriptions,
- Product identifiers,
- Similarity scores.

These identifiers served as source citations for retrieved information.

11. Limitations

Several limitations were identified during implementation.

- Product embeddings were generated primarily from product descriptions.
- Certification status and recall information were not directly incorporated into embeddings.
- The relatively small dataset limited the observable impact of indexing.
- Semantic retrieval quality depends heavily on the information included in the embedding process.

Despite these limitations, the system successfully demonstrated the integration of relational databases with vector-based retrieval.

12. Future Work

Future enhancements may include:

- Embedding information from certifications, inspections, and recalls jointly.
- Incorporating manufacturer and country information into semantic retrieval.
- Developing a graphical web interface.
- Supporting real-time vector updates.
- Evaluating alternative embedding models for improved retrieval quality.

These improvements could significantly enhance the accuracy and usability of the system.

13. Conclusion

This project successfully integrated traditional database concepts with modern AI techniques.

The Product Safety and Inspection Management System demonstrated:

- Effective relational database design,
- SQL-based analytical querying,
- Query optimization using indexes,
- Database automation through triggers,
- Retrieval-Augmented Generation using Sentence Transformers and Pinecone.

The resulting system provides both structured and semantic access to product safety information, illustrating how database technologies can be enhanced through artificial intelligence to support more intuitive information retrieval.

14. AI Usage Disclosure

Artificial intelligence tools were used to assist with debugging, implementation guidance, technical explanations, and report drafting. All generated outputs were reviewed, tested, validated, and adapted before inclusion in the final submission.